Networking request — email template

Fill the `<PLACEHOLDERS>`, delete the italic notes, pick the Zscaler option that matches your setup, and send. The three asks (certificate, DNS, Zscaler) can proceed in parallel; only the DNS **target value** waits on the gateway deploy, so the name can be reserved now.

Chicken-and-egg note: the certificate is signed against the FQDN only (not the ALB name), so it's actionable immediately. The CNAME target (`internal-*.elb.amazonaws.com`) only exists after we deploy the load balancer — we'll send that value the same day and ask you to populate the record then.

To: Network Engineering / PKI / Zscaler admins **Cc:** , **Subject:** Provisioning request — Claude gateway (GovCloud): enterprise cert, internal DNS CNAME, Zscaler access

Hi team,

We're standing up an internal-only application in AWS GovCloud (`us-gov-west-1`) — the Claude apps gateway for our developers. It sits behind an **internal** Application Load Balancer (no public exposure) and is reached only from our corporate network. To bring it online we need three things from you, detailed below. Happy to jump on a call to walk through any of them.

---

1. Enterprise-CA TLS certificate *(actionable now)*

The ALB terminates TLS with a certificate signed by our **internal** enterprise CA (a public CA can't issue for this name/deployment). We'll generate the key and CSR and handle the AWS import — we just need the CA to sign.

- **Subject / SAN:** `<GATEWAY_FQDN>` (e.g. `claude-gateway.<corp-domain>`) — the SAN must be **exactly** this FQDN.
- **Key / algorithm:** EC P-256 by default (CSR attached). *If our CA only issues RSA, tell us and we'll regenerate the CSR as RSA-2048/3072 — quick.*
- **Extended Key Usage:** must include **serverAuth**.
- **Return format (PEM):** the signed leaf certificate, and the CA chain as a separate file with **intermediates first, root last**.
- **Validity:** your standard server-cert lifetime is fine. Please note it — this cert is imported into AWS and does **not** auto-renew, so we'll set an expiry alarm and need a renewal owner on your side. *(Rotation is in-place and re-triggers a one-time client trust prompt, so we coordinate a heads-up before each renewal.)*

CSR: attached / provided separately as `<GATEWAY_FQDN>.csr`.

---

2. Internal DNS — CNAME record

A single CNAME in corporate DNS pointing our friendly name at the load balancer's AWS DNS name:

Record	Type	Target
`<GATEWAY_FQDN>`	CNAME	`<ALB_DNS_NAME>` — *we'll send this after deploy; format:* `internal-<name>-<id>.us-gov-west-1.elb.amazonaws.com`

Notes for your resolver config: - The ALB target is a **public** DNS record that returns **private** IP addresses — resolvable from any resolver, but routable only inside our VPC. So this is a normal CNAME in the corporate zone; **no** split-horizon / private hosted zone / conditional forwarder is required. - **Please confirm:** does our resolver enforce **DNS-rebinding protection** (stripping RFC1918/private answers out of public-zone responses)? If so it would break resolution of the ALB name and we'll need a small exception — worth checking before go-live.

---

3. Zscaler access

End users reach `<GATEWAY_FQDN>` from managed laptops. **Pick the option that matches how we deliver this app** (ZPA is preferred):

Option A — ZPA application segment *(preferred)*

- **App segment:** `<GATEWAY_FQDN>`, **TCP 443**, served by App Connectors that can route to our workload VPC (over the Transit Gateway).
- **Do NOT enable TLS inspection** on this segment — the client pins the gateway certificate's fingerprint, and interception breaks it.
- **Access policy:** scope to the developer group `<DEV_OKTA_OR_AD_GROUP>`.
- **App Connector DNS:** the connectors must be able to resolve `<GATEWAY_FQDN>`. On-prem connectors using AD DNS already can; connectors running **in an AWS VPC** need a Route 53 Resolver outbound rule forwarding our corporate zone to AD DNS (otherwise users see an unexplained ZPA timeout).

Option B — ZIA bypass *(if egress is via ZIA forward proxy instead)*

- **SSL-inspection exemption** for `<GATEWAY_FQDN>` (inspection breaks the certificate fingerprint pin).
- **Proxy/app bypass** for `<GATEWAY_FQDN>` so traffic doesn't egress via public Zscaler proxy IPs (the client's private-network check fails otherwise).

Why the no-inspection ask is load-bearing (impact if inspection stays on)

The Claude Code client **pins the SHA-256 fingerprint of the exact TLS leaf certificate** it receives from `<GATEWAY_FQDN>` (trust-on-first-use, per hostname, no configuration override). We publish the expected fingerprint to users, who confirm it at first login. Under TLS inspection the client receives the Zscaler-derived certificate instead, so:

- The fingerprint **never matches the published value** — users following instructions cannot log in at all; users who click through are being trained to accept the exact signal that detects a real MITM, which reduces the control's value to zero.
- Zscaler-derived leaf certificates are **not stable** (they change with intermediate rotation and can differ across service edges/nodes), so even accepted pins fail again at the next rotation — recurring "Claude Code stopped working" incidents at a cadence set by Zscaler cert policy, plus fail-closed startup errors for signed-in sessions.
- Inspection decrypts our developers' **source code, prompts, and gateway bearer tokens** on a hop from an internal client to an internal ALB — a new cleartext handling surface for our most sensitive content, with no monitoring benefit: the gateway already produces identity-stamped usage telemetry server-side, which is richer than anything a middlebox can see.

In short: with inspection enabled on this FQDN, the rollout ships with a security control that is either fully blocking or being systematically bypassed by users. There is no client-side workaround. For ZPA (option A) this only requires **not enabling** AppProtection/TLS inspection on the new segment; for ZIA (option B) it requires the exemption above.

Server-side egress: ALLOW + SSL-inspection exemption for the Okta issuer *(either option)*

- The gateway and Grafana containers (in the workload VPC) call `<OKTA_ISSUER_FQDN>` for OIDC discovery and OAuth **token exchange**. This is **server-originated** traffic: it carries no Zscaler user identity, so default ZIA policy both intercepts it AND blocks it (the observed failure sequence: a TLS failure from the derived cert, then **403 Forbidden** from policy once the cert was trusted).
- **Request (two parts, for the ZIA location/sub-location that carries the workload VPC's central egress):**
- **Allow** `<OKTA_ISSUER_FQDN>` (TCP 443) for unauthenticated / server-originated traffic from that egress source.
- **Exempt it from SSL inspection** on the same path.
- **Why exemption rather than trusting the inspection CA:** the token exchange carries the OIDC **client secret**; decrypting it means the IdP credential transits the inspection infrastructure. IdP endpoints are a standard SSL-inspection bypass category for exactly this reason.
- *(Fallback if the exemption is refused: we can bake the inspection root CA into the two container images —* `EXTRA_CA_CERT_PATH` *in the deploy tooling — but the policy ALLOW is required either way, and the exemption is the preferred posture.)*

Also needed for the offline client rollout (either option)

- **UNC file-share app segment:** the installer pulls `claude.exe` from `\\<FILE_SERVER_FQDN>\<share>` over **TCP 445** — this needs its **own** ZPA app segment (addressed by the fileserver FQDN), separate from the gateway segment above.

- **Machine Tunnel** *(if we push the client via Intune/SCCM in device/SYSTEM context)*: SYSTEM traffic isn't carried by the ZPA **user** tunnel, so the device-context install can't reach the UNC share without a Zscaler Machine Tunnel. *(Flag to the endpoint-management team if that's separate.)*

---

4. Related confirmations *(networking-adjacent — quick to overlook)*

If the workload VPC egresses through a **central inspection firewall** (TGW → central egress), please allow-list outbound from the gateway subnets to: - our **Okta issuer** host (`<OKTA_ISSUER_HOST>`) — this is the only internet-bound dependency; and - *(only if we don't use local VPC endpoints)* the AWS ECR / S3 / Amazon Managed Prometheus service domains for `us-gov-west-1`.

What we need back from you

1. **App Connector source CIDR(s)** — the IP ranges our connectors present to the ALB. We lock the load-balancer security group to exactly these, so we can't finish without them: `<PENDING — please provide>`.
2. **Certificate validity period** + who owns renewal on your side.
3. **DNS-rebinding-protection** answer (§2).
4. A rough **turnaround** / **ticket number** so we can sequence the deploy.

Thanks very much — this unblocks our test deployment. Reply-all or grab me at `<contact>`.

Best, `<name>`

Placeholder cheat-sheet (delete before sending)

Placeholder	Where it comes from
`<GATEWAY_FQDN>`	`GATEWAY_FQDN` in `scripts/deploy.env`
`<ALB_DNS_NAME>`	`AlbDnsName` output of the gateway stack (after `deploy-gateway.sh`)
`<DEV_OKTA_OR_AD_GROUP>`	the developer group gated for gateway access
`<FILE_SERVER_FQDN>` / `<share>`	where you stage `claude.exe` for the offline installer
`<OKTA_ISSUER_HOST>`	host part of `OKTA_ISSUER`
App Connector source CIDR	becomes `CLIENT_INGRESS_CIDR` in `deploy.env`